GUÍA OPERATIVA · SANDBOX

Arquitectura de referencia de DocuProcess Cloud

DocuProcess Cloud - Diagrama de componentes y justificacion tecnica RDS vs. DynamoDB

PROYECTO	DocuProcess Cloud
CURSO	Infraestructura III - Universidad ICESI
AUTORES	Daniela Castaño Moreno y Alejandro Pirez Garcia
VERSIÓN	1.0 · 13 de noviembre de 2026
REPOSITORIO	github.com/Alejandroperezgarc1a/docuprocess-cloud
REGIÓN	us-east-1

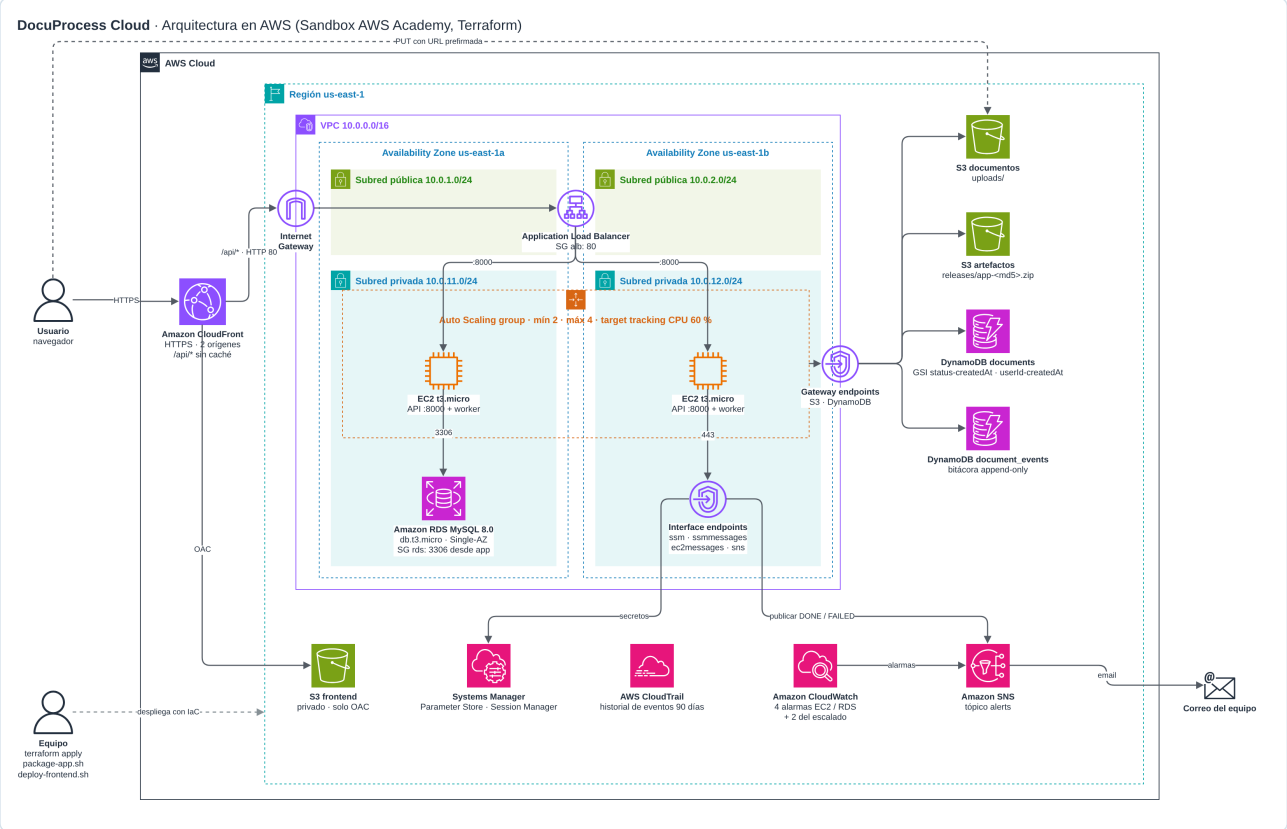
NAVEGACIÓN

Contenido

Los números de página corresponden al PDF. La guía rápida *despliegue.md* sigue disponible como referencia de bolsillo.

> Este documento tiene dos partes. La **vista general** (B11) describe la plataforma desplegada: diagrama de componentes, flujo de un documento, inventario de recursos por módulo de Terraform, permisos de la aplicación, restricciones del Sandbox y la justificación del diseño de datos (RDS frente a DynamoDB). Los **contratos de integración** (C1) son la fuente de verdad que comparten la API, el worker, el frontend y Terraform; todo cambio a un contrato se hace con un PR que actualice esa sección.

Diagrama de arquitectura en AWS



Vista con los íconos oficiales de AWS: región, VPC, las dos zonas de disponibilidad con sus subredes públicas y privadas, el Auto Scaling group y los servicios administrados. La fuente editable está en `img/docuprocess-aws.drawio` (se abre con draw.io / diagrams.net). El diagrama Mermaid de la sección siguiente muestra el mismo despliegue con el detalle de puertos y security groups.

Diagrama de componentes

Todo lo que aparece en el diagrama lo crea Terraform (`infra/`), salvo el usuario, el correo del equipo y el historial de eventos de CloudTrail, que AWS mantiene en cada cuenta sin recurso propio. Las líneas punteadas son tráfico de métricas o de subida directa.

Lectura del diagrama:

- **Un solo dominio público.** CloudFront sirve el frontend desde S3 (comportamiento por defecto, cacheado) y reenvía `/api/*` al ALB sin caché, con `Authorization` , `Content-Type` y `Origin` . El navegador siempre ve HTTPS; el tramo CloudFront → ALB es HTTP porque el Sandbox no permite dominio propio ni certificado ACM.
- **Subida directa.** El archivo no pasa por las instancias: el navegador hace `PUT` al bucket de documentos con la URL prefirmada que devuelve la API.
- **Red cerrada.** Las instancias y RDS viven en subredes privadas sin NAT ni IP pública. Todo el tráfico hacia AWS sale por endpoints: gateway para S3 y DynamoDB (gratis, rutas en la tabla privada) e

interface para SSM y SNS (ENI en cada subred privada con private DNS). Los repositorios `dnf` de Amazon Linux 2023 también se alcanzan por el gateway de S3.

- **Security groups encadenados.** `alb` (80 desde internet) → `app` (8000 solo desde `alb`) → `rds` (3306 solo desde `app`). `endpoints` acepta 443 desde la VPC. No hay puerto 22: la administración es por Session Manager.
- **Arranque de cada instancia.** El userData lee los secretos de Parameter Store, descarga el artefacto `releases/app-<md5>.zip`, instala `nodejs24`, arranca los servicios systemd de la API y el worker, y solo se da por lista cuando `GET /api/ready` llega a MySQL.

Flujo de un documento

Puntos clave del flujo:

- **Atomicidad estado + bitácora.** Cada transición escribe el documento y su evento en un `TransactWriteItems`; si una de las dos condiciones falla, no se escribe ninguna. Con `DDB_TRANSACTIONS=false` (respaldo del Sandbox) se escriben en secuencia.
- **Reclamo sin colas externas.** El `UpdateItem` condicional `#s = :pending` es la exclusión mutua: con 2 a 4 instancias consultando el mismo índice, solo una gana cada documento; las demás reciben `ConditionalCheckFailed` y siguen.
- **Cierre también condicional.** `DONE` y `FAILED` exigen `#s = :processing`, y el timeout de subida exige `#s = :pending`; un documento terminal nunca se reabre.
- **SNS es at-least-once.** La publicación se hace después de persistir la transición; si SNS falla, el estado ya está guardado y el error queda en el log del worker.

Inventario de componentes

Recurso	Módulo de Terraform	Propósito
VPC `10.0.0.0/16` con DNS habilitado	`network`	Red aislada de la aplicación; el DNS lo exige el private DNS de los interface endpoints
2 subredes públicas `10.0.1.0/24`, `10.0.2.0/24`	`network`	ALB en dos AZ
2 subredes privadas `10.0.11.0/24`, `10.0.12.0/24`	`network`	EC2, RDS e interface endpoints
Internet Gateway y tabla de rutas pública	`network`	Entrada y salida del ALB
Tabla de rutas privada compartida	`network`	Sin ruta a internet; rutas a los gateway endpoints
NAT Gateway y EIP	`network`	Opcional, desactivado (`enable_nat_gateway = false`)
Gateway endpoints `s3` y `dynamodb`	`network`	Acceso privado y gratuito a S3 (incluye repos de `dnf`) y DynamoDB
Interface endpoints `ssm`, `ssmmessages`, `ec2messages`, `sns`	`network`	Session Manager, Parameter Store y publicación SNS sin NAT
Security groups `alb`, `app`, `rds`, `endpoints`	`network`	Cadena 80 → 8000 → 3306 y 443 hacia los endpoints
Launch template AL2023, IMDSv2, `gp2` cifrado	`compute`	Receta de las instancias; el instance profile es el preexistente del Sandbox
Application Load Balancer y listener HTTP 80	`compute`	Punto de entrada de la API; origen `/api/*` de CloudFront

Recurso	Módulo de Terraform	Propósito
Target group `:8000`, health check `GET /api/health`	`compute`	Enruta a las instancias sanas; health superficial, no depende de RDS
Auto Scaling Group 2–4, health check ELB	`compute`	API y worker en dos AZ; el nombre lleva la versión del launch template para reemplazarse sin caída
Política target tracking CPU 60 %	`compute`	Escalado dinámico; AWS crea sus dos alarmas `AlarmHigh` y `AlarmLow`
Subnet group y RDS MySQL 8.0 `db.t3.micro`	`rds`	Usuarios y autenticación; Single-AZ, privado, `gp2` cifrado, backups de 1 día
Parámetros SecureString `/<name_prefix>/db_password` y `/jwt_secret`	raíz (`main.tf`)	Secretos fuera del userData y del launch template
Objeto `releases/app-<md5>.zip`	raíz (`main.tf`)	Artefacto de la app; un hash nuevo genera un ASG nuevo
Buckets `docs`, `frontend`, `artifacts` (`awscc_s3_bucket`)	`storage`	Documentos de usuario, sitio estático y releases
Bloqueo público, `BucketOwnerEnforced`, SSE-S3	`storage`	Los tres buckets privados y cifrados
CORS del bucket `docs`	`storage`	`PUT` prefirmado desde CloudFront y `localhost:5500`
Versionado del bucket `artifacts`	`storage`	Volver a un release anterior
Lifecycle `uploads/`	`storage`	Opcional, desactivado por defecto (`docs_lifecycle_expiration_days = 0`)
Bucket policies `docs` y `artifacts`	`storage`	Permisos del rol de la app sobre `uploads/*` y `releases/*`
`terraform_data` de vaciado	`storage`	Vacía los buckets con `scripts/empty-bucket.sh` antes de destruirlos
Tabla `documents` + 2 GSI, PITR, on-demand	`dynamodb`	Estado actual y cola del worker
Tabla `document_events` + 2 GSI, PITR, on-demand	`dynamodb`	Bitácora append-only
Resource policies de las dos tablas	`dynamodb`	Permisos de datos del rol; sin `UpdateItem` / `DeleteItem` en la bitácora
Tópico `alerts`, suscripción email y topic policy	`sns`	Avisos del worker y de las alarmas
Distribución CloudFront con 2 orígenes	`cdn`	`frontend-s3` (por defecto, caché) y `alb-api` (`/api/*`, sin caché)
Origin Access Control y bucket policy del frontend	`cdn`	Solo esta distribución puede leer el bucket del frontend
4 alarmas CloudWatch	`monitoring`	CPU del ASG > 80 %, CPU de RDS > 80 %, conexiones RDS > 50, espacio libre RDS < 2 GiB; acciones de alarma y OK a SNS
Trail, bucket dedicado y su política	`monitoring`	Solo con `cloudtrail_mode = "trail"`; en el Sandbox (`event-history`, por defecto) no se crean y la auditoría usa el historial de eventos

Permisos de la aplicación

Las instancias usan el instance profile del rol preexistente `AmazonSSMRoleForInstancesQuickSetup`, que solo trae permisos de SSM. Como el Sandbox no permite crear ni modificar roles, cada recurso de datos concede el acceso con una **política basada en el recurso** cuyo principal es el ARN del rol (`local.app_role_arn`).

Acción	Recurso	Concedida por	La usa
`s3:PutObject`	bucket `docs`, prefijo `uploads/*`	bucket policy `docs` (`storage`)	API: firma la URL prefirmada con las credenciales del rol
`s3:GetObject`	bucket `docs`, prefijo `uploads/*`	bucket policy `docs` (`storage`)	Worker: `HeadObject` y `GetObject` del archivo subido
`s3:GetObject`	bucket `artifacts`, prefijo `releases/*`	bucket policy `artifacts` (`storage`)	userData: descarga del artefacto
`dynamodb:GetItem`, `PutItem`, `UpdateItem`, `Query`	tabla `documents` e `index/*`	resource policy `documents` (`dynamodb`)	API: alta, detalle y lista; worker: cola, reclamo y cierre
`dynamodb:GetItem`, `PutItem`, `Query`	tabla `document_events` e `index/*`	resource policy `events` (`dynamodb`)	API y worker: eventos append-only y lectura de la bitácora
`sns:Publish`	tópico `alerts`	topic policy (`sns`)	Worker: aviso de `DONE` y `FAILED`
`ssm:GetParameter` con descifrado	`parameter/*`	política de identidad del rol, preexistente	userData: `db_password` y `jwt_secret`
Acciones del agente SSM	la instancia	`AmazonSSMManagedInstanceCore`, preexistente	Session Manager, sin SSH

`TransactWriteItems` no aparece porque no es una acción IAM: DynamoDB autoriza cada ítem de la transacción con su propia acción (`PutItem`, `UpdateItem`). La bitácora no concede `UpdateItem` ni `DeleteItem`, así que la inmutabilidad la impone AWS y no solo el código.

Otros principales con acceso por política de recurso:

Principal	Acción	Recurso	Concedida por
`cloudfront.amazonaws.com`, condición `AWS:SourceArn` = la distribución	`s3:GetObject`	bucket `frontend`	bucket policy del frontend (`cdn`)
`cloudwatch.amazonaws.com`, condición `aws:SourceAccount` = la cuenta	`sns:Publish`	tópico `alerts`	topic policy (`sns`)

Restricciones del Sandbox y solución

Restricción observada	Solución en el diseño
No se pueden crear roles ni políticas IAM; el rol de las EC2 solo tiene SSM	Permisos por políticas basadas en recursos (bucket, tabla, tópico); secretos en Parameter Store, que el rol sí puede leer
El instance profile tiene un nombre generado distinto en cada cuenta	Se descubre con `data "aws_iam_instance_profiles"` a partir del nombre del rol
`RunInstances` con volúmenes `gp3` es denegado	Disco raíz de EC2 y almacenamiento de RDS en `gp2`
`s3:GetBucketObjectLockConfiguration` es denegado y `aws_s3_bucket` falla	Buckets con `awscc_s3_bucket`, sub-recursos de `aws` y vaciado con `terraform_data` al destruir
`autoscaling:StartInstanceRefresh` es denegado por una SCP	Sin `instance_refresh`: el ASG lleva la versión del launch template en el nombre y se reemplaza con `create_before_destroy` + `wait_for_elb_capacity`
`cloudtrail:CreateTrail` es denegado; `LookupEvents` sí está permitido	`cloudtrail_mode = "event-history"` por defecto y `scripts/verify-cloudtrail.sh`; el modo `trail` queda listo para otra cuenta

Restricción observada	Solución en el diseño
`dynamodb:TransactWriteItems` no es una acción válida en una política	Se quitó de las resource policies; la transacción se autoriza por ítem
La topic policy no acepta `sns:*` y `s3:HeadObject` no es una acción válida	Acciones concretas en cada política (`sns:Publish` , `s3:GetObject`)
Sin dominio propio ni certificado ACM	Certificado por defecto de CloudFront; CloudFront → ALB en HTTP 80
Sin NAT no hay internet ni STS desde las instancias	Gateway e interface endpoints; `node_modules` empaquetados en el artefacto; la app no usa STS ni listados de cuenta
No se pueden crear llaves KMS propias	Cifrado con llaves administradas por AWS: SSE-S3, `aws/rds` , `aws/dynamodb` , `aws/ssm`
Tipos de instancia limitados	Validación en `instance_type` : solo `t2` / `t3` `micro` , `small` o `medium` ; por defecto `t3.micro`
Costo y tiempo de creación de RDS Multi-AZ	`multi_az = false` ; el subnet group ya cubre dos AZ para pasar a Multi-AZ sin rehacer la red

Justificación del diseño técnico: RDS vs. DynamoDB

La plataforma usa **dos motores de base de datos** porque resuelven problemas distintos. La regla de diseño es: **RDS para lo que exige integridad relacional; DynamoDB para lo que exige acceso por clave y escala horizontal; S3 para los bytes**.

Necesidad del sistema	Motor elegido	Por qué
Cuentas, correos únicos y autenticación	**RDS MySQL 8.0** (`db.t3.micro` , Single-AZ)	Datos relacionales con unicidad, transacciones ACID y consultas por correo; el esquema cambia poco y el volumen es pequeño
Estado de cada documento y cola del worker	**DynamoDB** (`documents` , on-demand)	Acceso por clave (`documentId`) y por índice (`userId` , `status`); escala sin administrar servidores y la carga es intermitente
Historial auditable de cambios de estado	**DynamoDB** (`document_events` , append-only)	Eventos inmutables ordenados por `documentId` + `eventAt` ; crecen sin migraciones ni joins
Archivos originales	**S3**	Objetos binarios de hasta 10 MB; no pertenecen a una base de datos

RDS MySQL: cuentas y autenticación

La tabla `users` necesita **unicidad del correo** (`UNIQUE(email)`), **transacciones ACID**, **consultas por atributo** (`WHERE email = ?`) e **integridad referencial** si se agregan tablas relacionadas. El correo es un atributo consultable, no una clave natural; en DynamoDB habría que mantener un ítem «correo → usuario» y coordinar dos escrituras por registro. Además, el login compara el hash de bcrypt después de buscar por correo: en SQL es una consulta trivial; en DynamoDB serían dos accesos por clave.

El esquema lo aplica la API al arrancar (`app/api/schema.sql`), no Terraform: Terraform crea la instancia, no las tablas. En el Sandbox la instancia es Single-AZ, `gp2` y `db.t3.micro` ; el subnet group cubre dos AZ para poder pasar a Multi-AZ sin rehacer la red.

DynamoDB: documentos, cola y bitácora

Los documentos tienen tres patrones de acceso conocidos de antemano:

1. **Por documento:** `GetItem(documentId)` para el detalle y para que el worker lo reclame.
2. **Por usuario:** `Query` del GSI `userId-createdAt` , más recientes primero (lista del frontend).

3. **Por estado:** `Query` del GSI `status-createdAt` para que el worker tome los `PENDING` más antiguos (cola lógica).

La bitácora añade un cuarto patrón: **por documento en orden cronológico** (`Query(documentId)` ascendente). Los GSI `userId-eventAt` y `day-eventAt` permiten auditar a un usuario entre documentos o un día completo sin `Scan`.

DynamoDB encaja porque el estado del documento es **variable en la práctica**: `result` cambia según el tipo (páginas para PDF, ancho/alto para imágenes) y `error` solo existe en `FAILED`. En SQL serían columnas nulas o una tabla de atributos; en DynamoDB es un mapa opcional. La cola se resuelve con el GSI y un `UpdateItem` condicional (`#s = :pending`), sin bloquear filas ni abrir conexiones mientras se reclama. El historial crece de forma monotónica y el `PutItem` es O(1); una tabla de eventos en MySQL obligaría a administrar particiones, purgas y capacidad.

La tabla `document_events` es **append-only**: la política de recurso del rol no concede `UpdateItem` ni `DeleteItem`, así que la inmutabilidad la impone AWS y no solo el código. Estado y evento se escriben juntos con `TransactWriteItems`; con `DDB_TRANSACTIONS=false` (respaldo del Sandbox) se escriben en secuencia.

Comparación lado a lado

Criterio	RDS MySQL	DynamoDB
Modelo	Relacional, SQL, joins	Clave-valor/documento, API por ítem
Esquema	Fijo, migraciones	Flexible; solo se declaran claves e índices
Unicidad	`UNIQUE` , `PRIMARY KEY`	Condiciones (`attribute_not_exists`) o claves diseñadas
Transacciones	ACID completas	`TransactWriteItems` / `TransactGetItems` (hasta 100 ítems)
Consistencia	Fuerte por defecto	Eventual en GSIs; `ConsistentRead` en la tabla base
Escala	Vertical (clase de instancia)	Horizontal (particiones)
Capacidad	Aprovisionada	On-demand o aprovisionada
Consultas ad hoc	Sí (`WHERE` , `JOIN` , `GROUP BY`)	No: hay que diseñar índices o usar `Scan` (caro)
Backups	Automáticos + snapshots	PITR (35 días) + on-demand
Seguridad de acceso	SG + usuario/contraseña + TLS	Políticas de recurso/IAM + cifrado en reposo
Operación	Parches y ventanas de RDS	Totalmente gestionado, sin servidores
Límite por ítem	Filas sin límite práctico	400 KB
Costo en la demo	Instancia `db.t3.micro` encendida 24/7	Por petición (céntimos)
Ajuste al problema	Cuentas y autenticación	Estado, cola e historial

Por qué no una sola base

Todo en RDS es viable a esta escala, pero acopla la carga del worker a la de autenticación: el worker sondea la cola cada 5 segundos desde 2 a 4 instancias y compite por CPU y conexiones con el login, que ya es la operación cara (bcrypt costo 12). La cola requeriría `SELECT … FOR UPDATE SKIP LOCKED` con conexiones abiertas, el `result` variable serían columnas nulas, la inmutabilidad de la bitácora dependería de `GRANT`s que la propia API administra y una caída de RDS (Single-AZ) detendría también el procesamiento.

Todo en DynamoDB obligaría a resolver la unicidad del correo con un ítem sintético y transacciones, a buscar el usuario por un atributo que no es clave y a renunciar a joins y consultas ad hoc. El volumen de usuarios de la demo no necesita escala horizontal y el enunciado pide RDS para cuentas y autenticación.

Consecuencia aceptada: una caída de RDS solo afecta registro y login; los documentos, la cola y el historial siguen disponibles. El health check del ALB (`/api/health`) no depende de MySQL, así que el ASG no recicla instancias por una caída de la base. Si el volumen creciera, el siguiente paso sería S3 → SQS → worker, manteniendo DynamoDB como fuente de verdad y el `UpdateItem` condicional como garantía de idempotencia.

1. API: framework y ejecución

- **Express 5 sobre Node.js 24 + TypeScript**, escuchando en `0.0.0.0:8000` con 2 workers (cluster).
- Por qué Express y no Fastify:
  - La API es pequeña y el equipo ya conoce Express; Express 5 reenvía automáticamente los errores de handlers `async` .
  - Todas las dependencias son JavaScript puro (sin binarios nativos), lo que simplifica el empaquetado de A4.
  - El cluster de Node replica los 2 workers síncronos del diseño anterior.
- **Node.js 24** en las instancias (paquete `nodejs24` de Amazon Linux 2023).

2. Servicios en las instancias

Elemento	Valor
Código	`/opt/docuprocess/{api,worker}`
Entorno de ejecución	`/opt/docuprocess/{api,worker}/dist` + `node_modules` vendorizado
Configuración	`/etc/docuprocess.env` (permisos 600, dueño root)
Unidades systemd	`docuprocess-api.service` , `docuprocess-worker.service` (`EnvironmentFile=/etc/docuprocess.env` , `Restart=always`)
Arranque de la API	`node /opt/docuprocess/api/dist/server.js` con `WEB_CONCURRENCY=2`
Arranque del worker	`node /opt/docuprocess/worker/dist/worker.js` (un proceso por instancia)
Usuario del sistema	`docuprocess`
Logs	stdout/stderr → journald (`journalctl -u docuprocess-api` , `journalctl -u docuprocess-worker`)
Log del arranque	`/var/log/docuprocess-userdata.log`

3. Variables de entorno

Variable	Origen en AWS	Usa	Local (`.env`)
`DOCS_BUCKET`	`module.storage.docs_bucket_name`	API, worker	bucket real (obligatorio para la API)
`DDB_TABLE`	`module.dynamodb.table_name`	API, worker	`documents`
`DDB_EVENTS_TABLE`	`module.dynamodb.events_table_name`	API, worker	`document_events`

Variable	Origen en AWS	Usa	Local (`.env`)
`DDB_TRANSACTIONS`	`var.enable_dynamodb_transactions` (`true`)	API, worker	`true`
`SNS_TOPIC_ARN`	`module.sns.topic_arn`	worker	vacío (solo log)
`DB_HOST`	`module.rds.db_address`	API	`127.0.0.1`
`DB_PORT`	`module.rds.db_port` (opcional, 3306)	API	`3306`
`DB_NAME`	`var.db_name`	API	`docuprocess`
`DB_USER`	`var.db_username`	API	`docuadmin`
`DB_PASSWORD`	Parameter Store `/<name_prefix>/db_password` (SecureString, lo lee el userData)	API	`.env`
`JWT_SECRET`	Parameter Store `/<name_prefix>/jwt_secret` (SecureString, lo lee el userData)	API	`.env`
`AWS_REGION`	`var.aws_region`	API, worker	`us-east-1`
`ARTIFACTS_BUCKET`	`module.storage.artifacts_bucket_name`	userData	vacío
`DDB_ENDPOINT_URL`	**no se define**	API, worker	`http://localhost:8001`
`CORS_ALLOWED_ORIGINS`	**no se define** (mismo origen vía CloudFront)	API	`http://localhost:5500`
`WORKER_POLL_SECONDS`	`var.worker_poll_seconds` (5)	worker	`5`
`WORKER_BATCH_SIZE`	`var.worker_batch_size` (10)	worker	`10`
`UPLOAD_TIMEOUT_MINUTES`	`var.upload_timeout_minutes` (15)	worker	`15`
`MAX_FILE_MB`	`var.max_file_mb` (10)	worker	`10`
`LOG_LEVEL`	fijo `INFO` en AWS	worker	`INFO`

Reglas:

- La aplicación **falla al arrancar** si falta una variable obligatoria (configuración por entorno, al estilo 12-factor).
- `DOCS_BUCKET` no puede estar vacío: la API falla al arrancar.
- El AWS SDK v3 recibe `region: config.awsRegion` explícitamente.
- Si `DDB_ENDPOINT_URL` está definida, se pasa como `endpoint_url` al cliente de DynamoDB.
- `DDB_EVENTS_TABLE` es obligatoria: sin bitácora la API no arranca (falla rápido, no a medias).
- `DDB_TRANSACTIONS=false` es solo un **fallback del Sandbox**: escribe el documento y luego el evento sin atomicidad. En AWS normal se deja `true`.
- En AWS, las credenciales vienen del instance profile vía IMDSv2; nunca se escriben en código ni en archivos.

4. Ítem de DynamoDB (tabla `documents`)

Atributo	Tipo	Descripción
`documentId`	S (PK)	UUID v4
`userId`	S	`id` del usuario en MySQL, como string

Atributo	Tipo	Descripción
`filename`	S	Nombre original saneado
`contentType`	S	Tipo declarado por el cliente
`s3Key`	S	`uploads/{userId}/{documentId}/{filename}`
`status`	S	`PENDING`, `PROCESSING`, `DONE` o `FAILED`
`createdAt`	S	ISO-8601 UTC con milisegundos: `2026-11-05T14:03:22.123Z` (ordena bien como string)
`updatedAt`	S	Igual formato
`result`	M	Solo en `DONE`: `sizeBytes` (N), `mimeType` (S), `sha256` (S), `pages` (N, PDF), `width` y `height` (N, imágenes)
`error`	S	Solo en `FAILED`

Índices:

- `userId-createdAt`: la lista de documentos de un usuario, del más reciente al más antiguo.
- `status-createdAt`: el worker busca los `PENDING` más antiguos.

Ciclo de vida de un documento:

```
PENDING ──(worker reclama con update condicional)──▶ PROCESSING ──▶ DONE
   │                                                              └──▶ FAILED (error)
   └──(archivo no subido en 15 min)──▶ FAILED ("upload timeout")
```

`status` es palabra reservada de DynamoDB: siempre va en `ExpressionAttributeNames` (`#s`).

4.1. Bitácora auditable (`document_events`)

`documents` guarda el **estado actual** (se sobrescribe). El **historial** vive en una tabla separada, `document_events`, que es *append-only*: la política del rol solo permite `PutItem`, `GetItem` y `Query`; las transacciones se autorizan con las acciones de cada ítem. No puede actualizar ni borrar eventos.

Atributo	Tipo	Descripción
`documentId`	S (PK)	UUID del documento
`eventAt`	S (SK)	ISO-8601 UTC con ms + `#` + 6 hex: `2026-11-11T14:03:22.123Z#a1b2c3`. El sufijo evita colisiones en el mismo milisegundo
`eventId`	S	UUID v4 del evento
`userId`	S	Dueño del documento (string)
`eventDay`	S	`YYYY-MM-DD` UTC (clave del GSI `day-eventAt`)
`fromStatus`	S	Estado anterior; ausente en el evento de creación
`toStatus`	S	Estado nuevo
`actor`	S	`api`, `worker` o `system`
`reason`	S	Motivo legible (opcional)
`error`	S	Detalle del fallo (opcional)
`requestId`	S	Correlación con logs (opcional)
`schemaVersion`	N	Versión del esquema del evento (hoy `1`)

Índices: `userId-eventAt` (auditoría de un usuario entre documentos) y `day-eventAt` (auditoría global por día). La creación usa `TransactWriteItems`: el `Put` del documento y el `Put` del evento se confirman juntos o ninguno. La condición del documento (`attribute_not_exists(documentId)`) evita sobrescribir uno existente; la del evento (`attribute_not_exists(eventAt)`) protege su clave exacta y no limita los demás eventos del documento. El resto de transiciones las hará el worker con un `Update` condicional del documento (`#s = :from`) + `Put` del evento.

El worker escribe `PENDING→PROCESSING`, `PROCESSING→DONE`, `PROCESSING→FAILED` y `PENDING→FAILED` (timeout de subida) con `actor="worker"`. Si `DDB_TRANSACTIONS=true`, el `Update` del documento y el `Put` del evento van en un `TransactWriteItems`; si es `false`, se escriben secuencialmente como respaldo del Sandbox.

5. Clave en S3

`uploads/{userId}/{documentId}/{filename}`. El nombre se sanea: sin `/`, sin `\`, sin caracteres de control y con un máximo de 200 caracteres. El `documentId` en la ruta evita colisiones entre archivos con el mismo nombre.

6. Autenticación (JWT)

- HS256 firmado con `JWT_SECRET`. Claims: `sub` (userId como string), `email`, `iat` y `exp` (1 hora).
- Header: `Authorization: Bearer <token>`.
- Contraseñas con bcrypt (cost 12). Esto también es lo que genera carga de CPU en la prueba de A8.

7. API HTTP

Todas las rutas empiezan por `/api` (CloudFront enruta `/api/*` al ALB). JSON en request y response.

Método y ruta	Auth	Request	Respuesta exitosa	Errores
`GET /api/health`	No	—	`200 {"status":"ok"}` (no toca la base)	—
`GET /api/ready`	No	—	`200 {"status":"ready","checks":{"mysql":"ok"}}` (`SELECT 1` a MySQL)	503
`POST /api/auth/register`	No	`{email, password}` (≥ 8 caracteres)	`201 {id, email}`	400, 409
`POST /api/auth/login`	No	`{email, password}`	`200 {token, expiresIn: 3600}`	400, 401
`POST /api/documents`	Sí	`{filename, contentType, size}`	`201 {documentId, uploadUrl, method:"PUT", headers:{"Content-Type"}, key, expiresIn: 900}`	400, 401
`GET /api/documents`	Sí	—	`200 {items:[…]}` (máx. 50, más recientes primero)	401
`GET /api/documents/{id}`	Sí	—	`200 {…ítem…}`	401, 404 (no existe o es de otro usuario)
`GET /api/documents/{id}/events`	Sí	—	`200 {items:[…]}` (bitácora ascendente, máx. 200)	401, 404, 503

Formato de error: `{"error": "<codigo>", "message": "<texto legible>"}`.

8. Archivos permitidos

`application/pdf`, `image/png`, `image/jpeg` y `text/plain`, con un máximo de **10 MB**. Lo valida el frontend (B8), la API sobre lo declarado (A6) y el worker sobre el archivo real (B5). Una URL prefirmada de tipo PUT no puede imponer el tamaño, por eso el worker es la última barrera. El worker detecta el MIME por magic bytes (`file-type`) y, si no hay firma conocida, acepta UTF-8 válido como `text/plain`; rechaza el resto.

9. Notificación SNS (worker)

10. Nombres en Terraform

Locals compartidos (raíz): `name_prefix = "${project_name}-${environment}"` (por defecto `docuprocess-dev`), `instance_profile_name` (descubierto) y `app_role_arn` (construido).

Variables raíz: ver `infra/variables.tf` (C2). Las principales son `aws_region`, `project_name`, `environment`, `azs`, `vpc_cidr`, `public_subnet_cidrs`, `private_subnet_cidrs`, `enable_nat_gateway`, `enable_interface_endpoints`, `instance_type`, `instance_role_name`, `asg_min_size`, `asg_max_size`, `asg_desired_capacity`, `cpu_target_value`, `root_volume_size`, `db_name`, `db_username`, `db_password`, `jwt_secret`, `alert_email`, `dynamodb_table_name`, `document_events_table_name`, `enable_dynamodb_transactions`, `enable_point_in_time_recovery`, `worker_poll_seconds`, `worker_batch_size`, `upload_timeout_minutes`, `max_file_mb`, `app_artifact_path`, `cors_allowed_origins`, `force_destroy_buckets` y `cloudtrail_mode`.

Outputs por módulo (nombres exactos):

Módulo	Outputs
network	`vpc_id`, `public_subnet_ids`, `private_subnet_ids`, `private_route_table_ids`, `alb_security_group_id`, `app_security_group_id`, `rds_security_group_id`
compute	`alb_dns_name`, `alb_arn_suffix`, `target_group_arn_suffix`, `asg_name`, `launch_template_id`
rds	`db_address`, `db_port`, `db_instance_identifier`
storage	`docs_bucket_name`, `docs_bucket_arn`, `frontend_bucket_name`, `frontend_bucket_arn`, `frontend_bucket_regional_domain_name`, `artifacts_bucket_name`, `artifacts_bucket_arn`
dynamodb	`table_name`, `table_arn`, `events_table_name`, `events_table_arn`
sns	`topic_arn`, `topic_name`
cdn	`cloudfront_domain_name`, `distribution_id`, `distribution_arn`
monitoring	`alarm_names`, `cloudtrail_bucket_name`, `cloudtrail_arn`, `cloudtrail_audit_mode`

Outputs raíz: `alb_dns_name`, `cloudfront_domain_name`, `cloudfront_distribution_id`, `app_url`, `docs_bucket_name`, `frontend_bucket_name`, `artifacts_bucket_name`, `dynamodb_table_name`, `dynamodb_events_table_name`, `sns_topic_arn`, `rds_endpoint`, `asg_name`, `vpc_id`, `alarm_names`, `cloudtrail_bucket_name`, `cloudtrail_arn` y `cloudtrail_audit_mode`.

11. Permisos de la aplicación: políticas basadas en recursos

El rol de las instancias (`AmazonSSMRoleForInstancesQuickSetup`) **solo tiene permisos de SSM** (ver la sección 12). Cada módulo de datos recibe `app_role_arn` y concede lo mínimo **en su propio recurso**:

Recurso	Política	Acciones concedidas al rol
Bucket de documentos	bucket policy	`s3:GetObject` y `s3:PutObject` sobre `uploads/*`
Bucket de artefactos	bucket policy	`s3:GetObject` sobre `releases/*`
Tabla `documents` e índices	DynamoDB resource policy	`GetItem`, `PutItem`, `UpdateItem`, `Query` (las transacciones usan estas acciones)
Tabla `document_events` e índices	DynamoDB resource policy	`GetItem`, `PutItem`, `Query` (sin `UpdateItem` / `DeleteItem`: append-only; las transacciones usan estas acciones)
Tópico de alertas	topic policy	`sns:Publish` (y `sns:Publish` para `cloudwatch.amazonaws.com`, por las alarmas)
Parámetros `/<name_prefix>/*`	ya cubierto por el rol (`ssm:*` sobre parámetros)	`ssm:GetParameter`

La app **no** debe usar operaciones de listado de cuenta (`ListBuckets`, `ListTables`, `ListTopics`) ni STS: no están permitidas ni son alcanzables sin NAT.

Hallazgo	Consecuencia en el diseño
El rol de las EC2 solo tiene `AmazonSSMManagedInstanceCore` + `ssm:*` sobre `parameter/*`	Permisos de la app por políticas basadas en recursos (sección 11); secretos en Parameter Store
El instance profile tiene un nombre generado (`…-SsmRoleInstanceProfile-…`) distinto en cada cuenta	Se descubre con `data "aws_iam_instance_profiles"` a partir del nombre del rol; nunca `data "aws_iam_role"`
`RunInstances` con volúmenes **gp3** → *explicit deny* (la AMI AL2023 trae gp3)	Disco raíz forzado a **gp2**
`s3:GetBucketObjectLockConfiguration` → *explicit deny*; `aws_s3_bucket` falla (providers 4.x, 5.x y 6.x)	Buckets con `awscc_s3_bucket` + sub-recursos de `aws` + vaciado al destruir (`scripts/empty-bucket.sh`)
`autoscaling:StartInstanceRefresh` → *explicit deny* de una SCP (A4, 2026-11-09)	Sin `instance_refresh`: el ASG lleva la versión del launch template en el nombre y se reemplaza sin caída (`create_before_destroy` + `wait_for_elb_capacity`; no es azul/verde estricto: los dos ASG comparten el target group durante el cambio)
Sin NAT no hay STS ni internet; SSM, S3, DynamoDB, SNS y `dnf` funcionan por endpoints	`enable_nat_gateway = false`; dependencias de Node.js (`node_modules`) empaquetadas en el artefacto
La topic policy de SNS no acepta `sns:*`; `s3:HeadObject` no es una acción válida	Listar acciones concretas en las políticas
`dynamodb:TransactWriteItems` no es una acción IAM válida (Sandbox, B3, 2026-11-12)	No incluirla en las políticas: DynamoDB autoriza la transacción con las acciones de cada ítem
Contingencia (no observada aún): una SCP podría denegar las escrituras transaccionales (acciones de ítem como `PutItem` / `UpdateItem`) o `dynamodb:PutResourcePolicy`	`enable_dynamodb_transactions = false` escribe estado y evento por separado; sin `PutResourcePolicy` no hay forma de dar datos al rol (el Sandbox no permite crear roles) y habría que escalar el permiso

CloudTrail en el Sandbox (B10, 2026-11-13): el rol `voclabs` no permite `cloudtrail:CreateTrail` (error de política identity-based), pero sí `cloudtrail:LookupEvents`. Por eso `module.monitoring` usa por defecto `cloudtrail_mode = "event-history"`: la auditoría se consulta con el historial de eventos de CloudTrail (90 días) mediante `scripts/verify-cloudtrail.sh`. El modo `"trail"` crea el bucket dedicado, la política S3 y `aws_cloudtrail`; queda listo para una cuenta con permiso de creación de trails.

```
cp .env.example .env            # completar contraseñas y JWT_SECRET
docker compose up -d
docker compose ps               # mysql: healthy, dynamodb: running
```

```
# Crear la tabla en DynamoDB Local (mismo esquema que el módulo dynamodb)
AWS_ACCESS_KEY_ID=local AWS_SECRET_ACCESS_KEY=local \
aws dynamodb create-table --endpoint-url http://localhost:8001 --region us-east-1 \
  --table-name documents --billing-mode PAY_PER_REQUEST \
  --attribute-definitions \
      AttributeName=documentId,AttributeType=S AttributeName=userId,AttributeType=S \
      AttributeName=status,AttributeType=S AttributeName=createdAt,AttributeType=S \
  --key-schema AttributeName=documentId,KeyType=HASH \
  --global-secondary-indexes \
    '[{"IndexName":"userId-createdAt","KeySchema":[{"AttributeName":"userId","KeyType":"HASH"},
{"AttributeName":"createdAt","KeyType":"RANGE"}],"Projection":{"ProjectionType":"ALL"}},
      {"IndexName":"status-createdAt","KeySchema":[{"AttributeName":"status","KeyType":"HASH"},
{"AttributeName":"createdAt","KeyType":"RANGE"}],"Projection":{"ProjectionType":"ALL"}}]'

# Bitácora append-only (mismo esquema que aws_dynamodb_table.events)
AWS_ACCESS_KEY_ID=local AWS_SECRET_ACCESS_KEY=local \
aws dynamodb create-table --endpoint-url http://localhost:8001 --region us-east-1 \
  --table-name document_events --billing-mode PAY_PER_REQUEST \
  --attribute-definitions \
      AttributeName=documentId,AttributeType=S AttributeName=eventAt,AttributeType=S \
      AttributeName=userId,AttributeType=S AttributeName=eventDay,AttributeType=S \
  --key-schema AttributeName=documentId,KeyType=HASH AttributeName=eventAt,KeyType=RANGE \
  --global-secondary-indexes \
    '[{"IndexName":"userId-eventAt","KeySchema":[{"AttributeName":"userId","KeyType":"HASH"},
{"AttributeName":"eventAt","KeyType":"RANGE"}],"Projection":{"ProjectionType":"ALL"}},
      {"IndexName":"day-eventAt","KeySchema":[{"AttributeName":"eventDay","KeyType":"HASH"},
{"AttributeName":"eventAt","KeyType":"RANGE"}],"Projection":{"ProjectionType":"ALL"}}]'

# Worker en local (DynamoDB Local + bucket real; SNS vacío escribe en el log)
cd app/worker && npm ci && npm run build && node dist/worker.js

docker compose down             # detener (DynamoDB Local es en memoria: las tablas se pierden)
docker compose down -v          # además borrar los datos de MySQL
```

DynamoDB Local acepta cualquier credencial; las de arriba (`local`) evitan usar por accidente las del Sandbox.

14. Integración frontend/API

- **Desarrollo:** Vite sirve el frontend en `http://localhost:5500` y reenvía `/api` a `http://localhost:8000` ; el navegador ve un solo origen y no requiere CORS para la API.
- **Producción:** CloudFront sirve `frontend/dist/` en la raíz y enruta `/api/*` al ALB. El frontend usa rutas relativas. El behavior de `/api/*` no debe cachear y debe reenviar `Authorization` , `Content-Type` y `Origin` .
- **Build y configuración:** `npm ci && npm run build` en `frontend/` ; `prebuild` genera `frontend/public/config.js` desde `config.example.js` con `apiBaseUrl: ""` para usar rutas relativas. Se publica `frontend/dist/` , no el código fuente. `./scripts/deploy-frontend.sh` construye, sincroniza el bucket, aplica caché (`no-cache` para `index.html` y `config.js` , larga para assets con hash) e invalida CloudFront con `/*` .
- **CORS del bucket de documentos:** el `PUT` de la URL prefirmada va directo del navegador a S3. El bucket debe permitir `PUT` , `GET` y `HEAD` desde `http://localhost:5500` , `http://127.0.0.1:5500` y el dominio de CloudFront, con `AllowedHeaders: ["Content-Type"]` .
- **Orígenes CORS en B2/B7:** el módulo usa orígenes explícitos (`localhost:5500` y `127.0.0.1:5500`). B7 añade `https://${module.cdn.cloudfront_domain_name}` a `module.storage.cors_allowed_origins` conservando los orígenes locales para el desarrollo. No hay ciclo: el bucket (salida de storage) no depende de la regla CORS; la regla depende del dominio de CloudFront y CloudFront depende del bucket.

- **Health del ALB:** el target group debe apuntar a `GET /api/health` (superficial: una caída de RDS no recicla las instancias).
- **Readiness al arrancar:** el userData espera `GET /api/ready` (MySQL) antes de dar por lista la instancia. Si no responde, apaga la API: la instancia nunca queda healthy, el despliegue no avanza y el ASG anterior sigue atendiendo.

El CORS del bucket y el behavior de CloudFront se implementan en B2/B7; el health check en A3.